RUNBOOKGEN AI: AI-Powered IT Troubleshooting Runbook Generator

Project submitted to the

SRM University – AP, Andhra Pradesh

for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

Computer Science and

Engineering School of

Engineering and Sciences

Submitted by

V. Amrutha Preethi – AP23110010915
V. Likhita – AP23110010931
D. Karunya – AP23110010898

Under the Guidance of
Ravi Teja

SRM University–AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240

[April, 2026]

Project Description:

RunbookGen AI is an intelligent, AI-assisted IT troubleshooting system designed to automate the creation of structured runbooks from user-reported technical issues. In modern IT environments, engineers often face difficulties in diagnosing problems due to fragmented documentation, lack of standard procedures, and time constraints. Traditional troubleshooting involves manual searching through logs and resources, making it time-consuming and error-prone.

RunbookGen AI addresses these challenges by converting natural language problem descriptions into clear, step-by-step troubleshooting guides. It combines document retrieval techniques with rule-based intelligence and optional AI support to generate accurate and structured runbooks.

The system processes user queries such as "Apache server is not starting" through a multi-stage pipeline. It first performs semantic document retrieval using FAISS and embeddings to fetch relevant technical information. Then, a rule-based engine analyzes the issue type (such as web server, network, or storage) and determines the appropriate troubleshooting approach.

Based on this, the system generates a structured runbook including:

- Problem Summary
- Possible Causes
- Troubleshooting Steps
- Commands
- Risk Warnings
- Rollback Plan
- Validation Steps

This structured format ensures clarity and consistency, making it especially useful for junior engineers.

Additionally, the system includes features such as history tracking, runbook export, and user authentication, improving usability and enabling knowledge reuse. The frontend is designed using HTML and CSS for a clean interface, while the backend is implemented using Flask to manage logic and integration.

RunbookGen AI is useful for IT support teams, DevOps environments, and system administration tasks. It reduces troubleshooting time, minimizes errors, and standardizes the problem-solving process.

In conclusion, the system demonstrates how retrieval-based methods and automation can improve efficiency and reliability in IT operations, serving as a practical solution for real-world technical challenges.

Objectives

Primary Objective:

- To design and develop an intelligent system that automatically generates structured IT troubleshooting runbooks from user-reported issues.

Secondary Objectives:

1. Automation of Troubleshooting
    - To reduce manual effort by automating the process of diagnosing and resolving technical issues.
2. Standardization of Solutions
    - To provide consistent and well-structured troubleshooting steps for different types of system problems.

3. Efficient Information Retrieval
    - To use retrieval techniques (FAISS and embeddings) to fetch relevant technical documentation for accurate solutions.

4. User-Friendly Interface
    - To develop a simple and clean interface that allows users to easily input issues and view results.
5. Knowledge Reusability
    - To store generated runbooks in a database for future reference and reuse.
6. Support for Beginners
    - To assist junior engineers by providing clear, step-by-step guidance for troubleshooting.
7. Improved Accuracy and Reliability
    - To minimize errors by combining rule-based logic with relevant documentation.
8. Export and Documentation Support
    - To enable users to download runbooks for reporting, sharing, or documentation purposes.

Use Case Scenarios

Scenario 1: IT Issue Troubleshooting

In this scenario, a user enters a technical issue such as “Apache server not starting” into the system. RunbookGen AI processes the input, retrieves relevant documentation using semantic search, and analyzes the issue type. Based on this, it generates a structured runbook that includes possible causes, troubleshooting steps, commands, and validation procedures. This helps the user quickly identify and resolve the issue in a systematic way.

Scenario 2: Learning Support for Beginners

This scenario focuses on helping junior engineers and students. Users can input different system-related problems and observe how the system generates detailed troubleshooting steps. By following these structured solutions, users can understand the correct approach to diagnosing issues and improve their practical knowledge in system administration.

Scenario 3: Knowledge Reuse and History Management

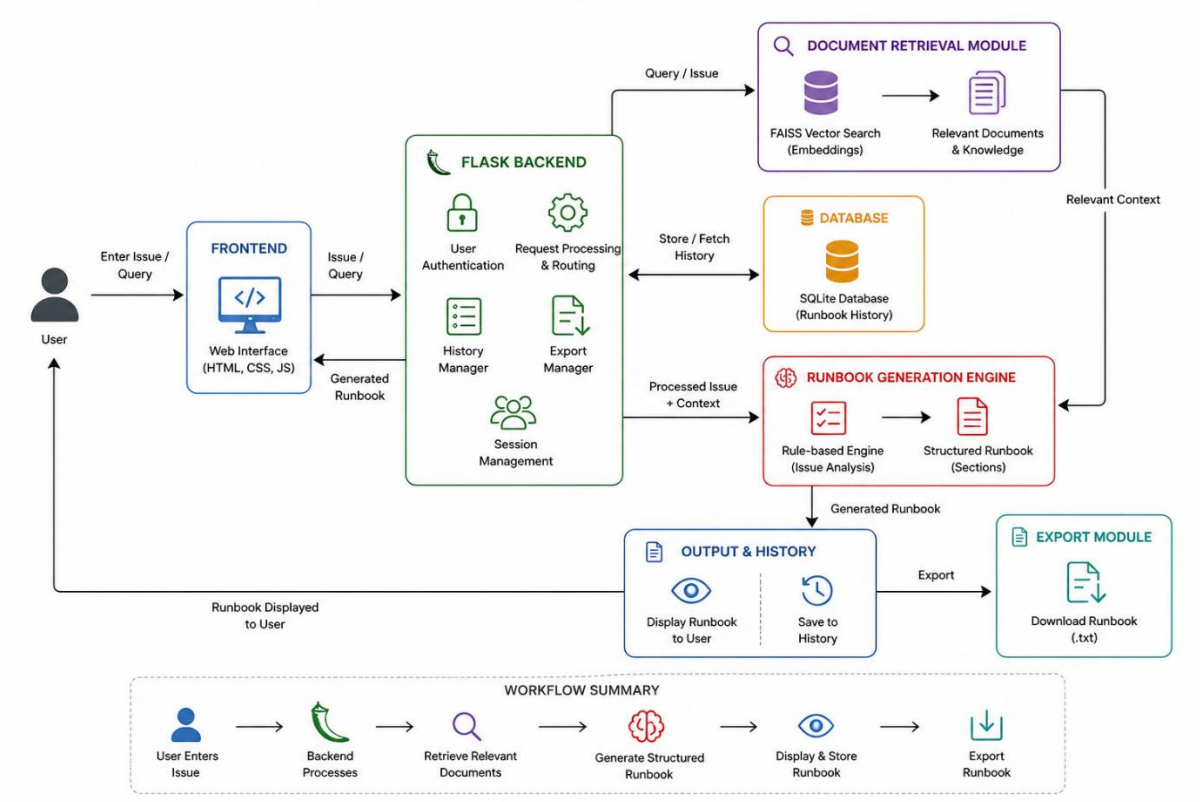
In this scenario, the system stores all previously generated runbooks in a database. Users can access the history section to view past solutions. This reduces repeated effort, saves time, and allows users to reuse solutions for similar problems without generating them again.

Scenario 4: Documentation and Reporting

Here, users can export the generated runbooks in text format. This feature is useful for maintaining documentation, sharing troubleshooting steps with team members, and including them in reports or audits. It ensures that solutions are properly recorded and easily accessible.

Architecture Overview

RunbookGen AI follows a simple modular architecture. The user enters an IT issue through the web interface. The Flask backend receives the query and sends it to the document retrieval module, which uses FAISS and embeddings to fetch relevant technical content. Then, the runbook generation engine creates a structured troubleshooting guide. The final output is displayed to the user and stored in the SQLite database for future reference.



Core Technologies

The development of RunbookGen AI involves the use of the following technologies:

Frontend Technologies

- HTML: Used to structure the web pages and input forms
- CSS: Used to design a clean and responsive user interface
- JavaScript (optional): Enhances interactivity and dynamic content handling

Backend Technologies

- Python: Core programming language used for implementing system logic
- Flask: Lightweight web framework used for handling routing, requests, and backend processing

AI & Retrieval Technologies

- FAISS (Facebook AI Similarity Search): Used for efficient document retrieval
- Sentence Transformers: Used to convert text into embeddings for semantic search

Database

- SQLite: Used to store generated runbooks and maintain history records

Optional AI Integration

- Ollama / LLM Models: Used for advanced AI-based runbook generation (optional feature)

Component-Wise Architecture

The RunbookGen AI system is divided into multiple components, each responsible for a specific functionality. This modular design ensures efficient processing, easy maintenance, and scalability.

1. Frontend UI:
   The frontend is developed using HTML and CSS. It provides an interface where users can enter IT issues, view generated runbooks, and access history. It ensures a clean, user-friendly, and responsive experience.

2. Flask Backend:
   The backend is implemented using Flask, which handles user requests, routing, and communication between different modules. It processes user input, manages sessions, and controls the overall workflow of the system.

3. Document Retrieval Module:
   This module uses FAISS and sentence-transformer embeddings to retrieve relevant technical documentation based on the user query. It ensures that the generated runbooks are based on meaningful and context-aware information.

4. Runbook Generation Engine:
   This component analyzes the issue type using rule-based logic (with optional AI support) and generates structured troubleshooting steps. It creates organized sections such as causes, commands, and validation steps.

5. Database Layer (SQLite):
   SQLite is used to store generated runbooks and maintain history records. It allows users to revisit past solutions and supports efficient data storage and retrieval.

6. History Manager:
   This component retrieves stored runbooks from the database and displays them to the user. It enables knowledge reuse and reduces repeated effort.

7. Export Module:
   This module allows users to download generated runbooks in text format. It is useful for documentation, sharing, and reporting purposes.

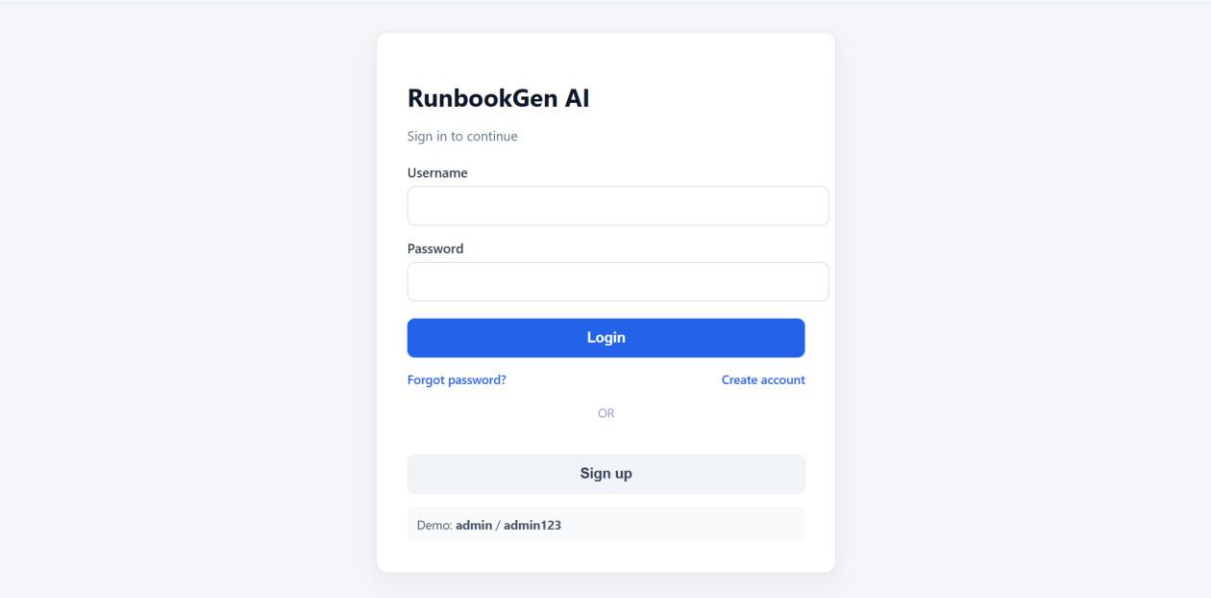
8. Session Management:
   This component manages user authentication and session handling, ensuring secure access to the system.

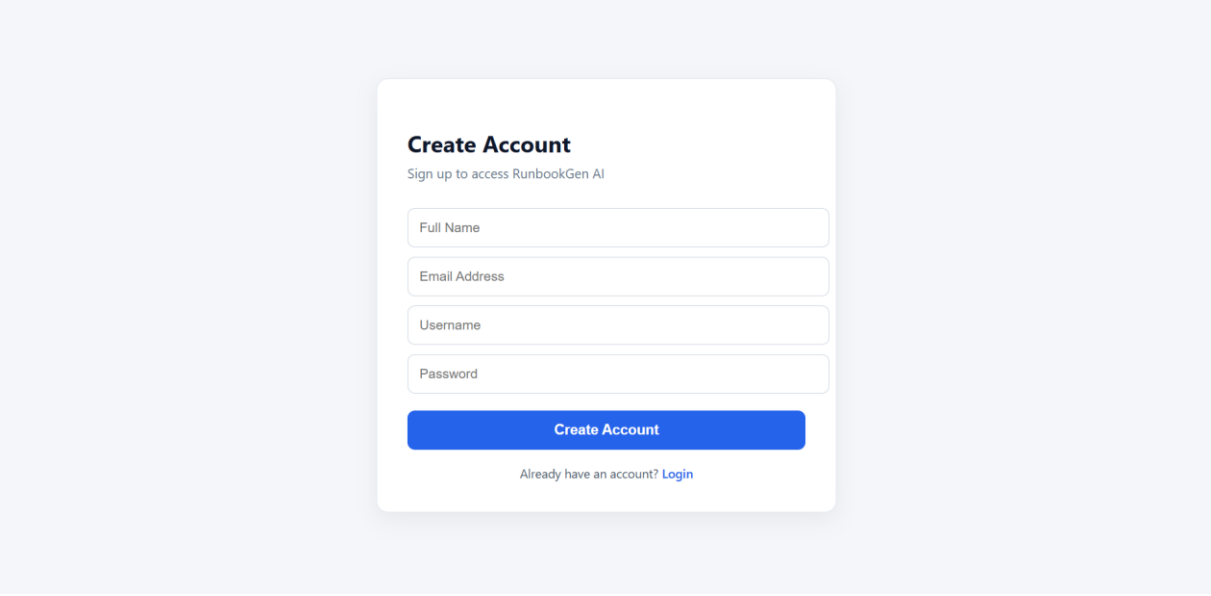
Project Workflow

The workflow of RunbookGen AI describes how the system processes a user query and generates a structured troubleshooting runbook.

Step 1: User Login

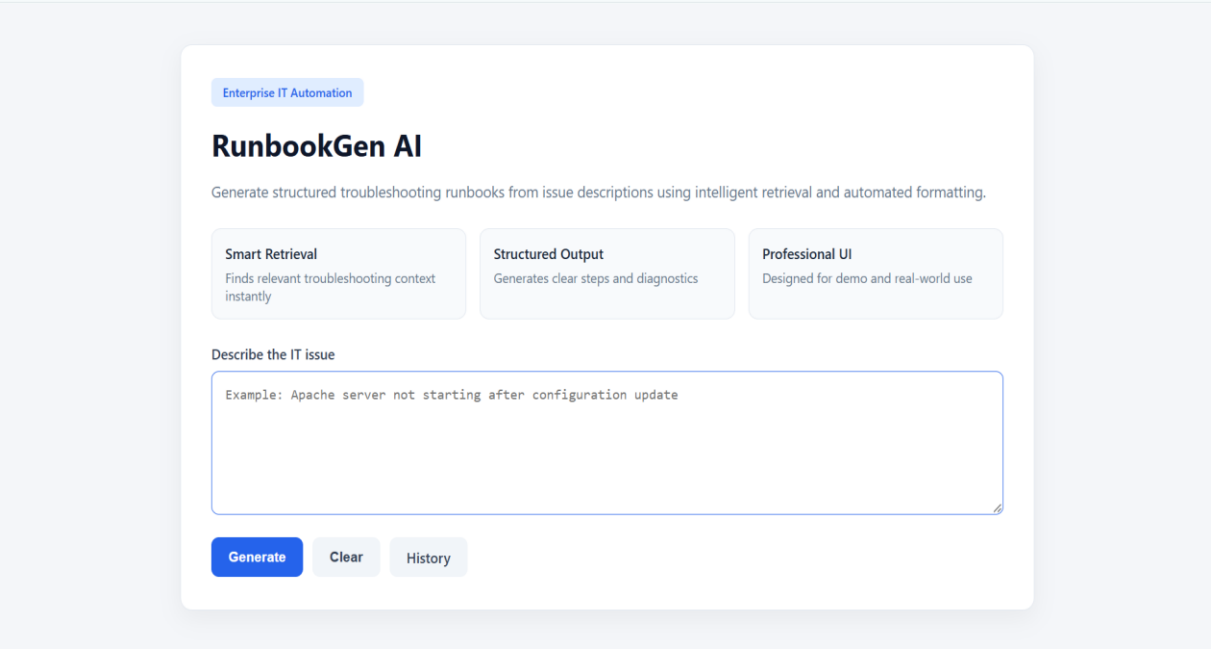
The user logs into the system using valid credentials. This ensures secure access and session management.





Step 2: User Input

The user enters a system-related issue through the web interface, such as “Apache server not starting”. This input acts as the starting point for the workflow.



Step 3: Backend Processing

The Flask backend receives the user query and performs initial processing. It manages routing, validates input, and prepares the query for further analysis.

app.py

```python
from flask import Flask, render_template, request, Response, redirect, url_for, session

from generator import generate_runbook
from retriever import retrieve_documents
from models import db, RunbookHistory

app = Flask(__name__)
app.secret_key = "runbookgen_secret_key"

USERNAME = "admin"
PASSWORD = "admin123"

app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///runbooks.db"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

db.init_app(app)

with app.app_context():
    db.create_all()


@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")

        if username == USERNAME and password == PASSWORD:
            session["logged_in"] = True
            return redirect(url_for("index"))
        else:
            return render_template("login.html", error="Invalid username or password")
```

app.py

```
23  def login():
34      return render_template("login.html")
35
36
37  @app.route("/logout")
38  def logout():
39      session.clear()
40      return redirect(url_for("login"))
41
42
43  @app.route("/", methods=["GET"])
44  def index():
45      if not session.get("logged_in"):
46          return redirect(url_for("login"))
47
48      return render_template("index.html")
49
50
51  @app.route("/generate", methods=["POST"])
52  def generate():
53      if not session.get("logged_in"):
54          return redirect(url_for("login"))
55
56      query = request.form.get("issue", "").strip()
57      use_llm = False
58
59      if not query:
60          return render_template(
61              "index.html",
62              error="Please enter a system issue, for example: Apache service not starting on RHEL.",
63          )
```

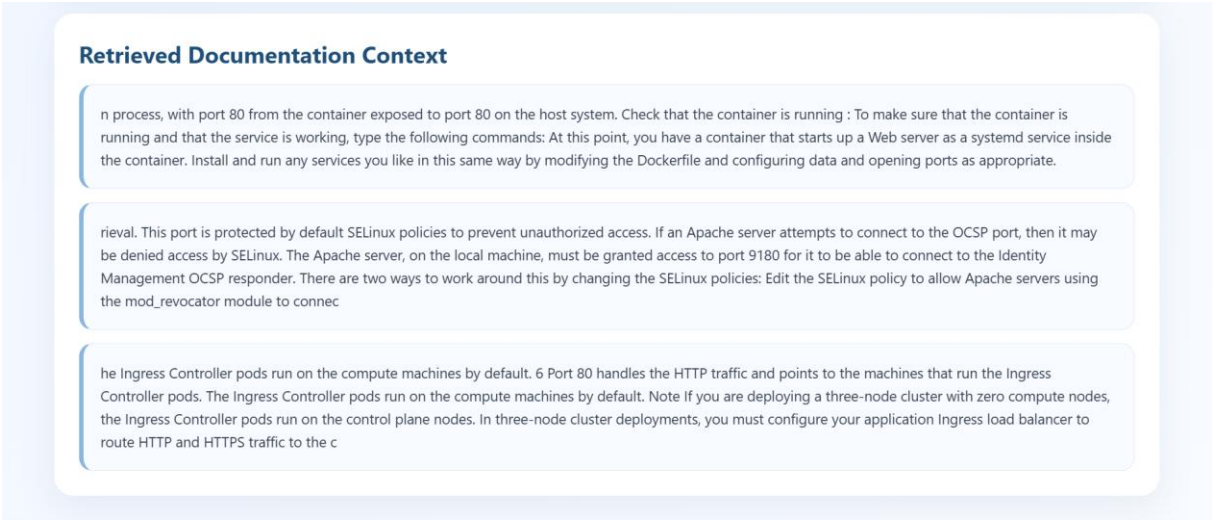
app.py

```
52  def generate():
65      try:
66          docs = retrieve_documents(query, top_k=3, max_results=5)
67          result, mode = generate_runbook(query, docs, use_llm=use_llm)
68
69          history = RunbookHistory(issue=query, result=result, mode=mode)
70          db.session.add(history)
71          db.session.commit()
72
73      except Exception as exc:
74          docs = []
75          mode = "Error"
76          result = f"Error while generating runbook: {exc}"
77
78      return render_template("result.html", query=query, result=result, docs=docs, mode=mode)
79
80
81  @app.route("/history")
82  def history():
83      if not session.get("logged_in"):
84          return redirect(url_for("login"))
85
86      items = RunbookHistory.query.order_by(RunbookHistory.created_at.desc()).all()
87      return render_template("history.html", items=items)
88
89
90  @app.route("/export", methods=["POST"])
91  def export():
92      if not session.get("logged_in"):
93          return redirect(url_for("login"))
94
95      content = request.form.get("content", "")
```

Step 4: Document Retrieval

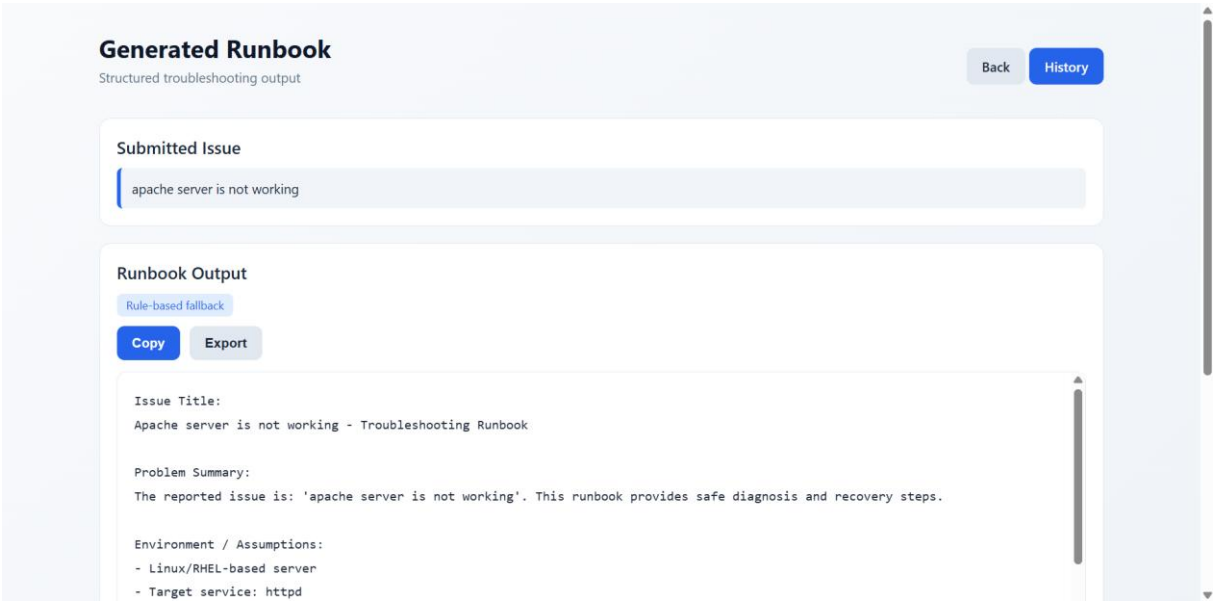
The system uses FAISS and embeddings to retrieve relevant technical documentation based on the query. This ensures that the solution is based on real and meaningful context.





Step 5: Issue Analysis & Runbook Generation

The system analyzes the issue and generates a structured runbook containing causes, steps, commands, and validation procedures.



- Target service: httpd
- Engineer has sudo/root access

Possible Causes:
- Service is stopped or failed
- Syntax error in web server configuration
- Port 80/443 conflict
- Firewall rule missing
- SELinux policy blocking access

Documentation-Based Hints:
- Install and run any services you like in this same way by modifying the Dockerfile and configuring data and opening ports as appropriate.

Step-by-Step Troubleshooting:
1. Confirm exact error, host, service name, and recent changes.
2. Check service status before changing anything.
3. Review recent logs and identify the first useful error.
4. Validate configuration files before restart.
5. Check firewall, SELinux, ports, disk space, and permissions.
6. Apply the smallest safe fix and record it.
7. Restart only the affected service after impact check.

Runbook Output

Rule-based fallback

Copy Export

8. Validate from local and client side.

Commands:
- systemctl status httpd
- journalctl -u httpd -n 80 --no-pager
- apachectl configtest # Apache/httpd only
- ss -tulnp | grep -E ':80|:443'
- firewall-cmd --list-all
- systemctl restart httpd

Risk Warnings:
- Restarting services may interrupt users.
- Firewall or SELinux changes can block or expose services.
- Do not delete files or change permissions without backup/approval.

Rollback Plan:
- Backup config files before editing.
- Revert changed files/settings if validation fails.
- Restart httpd again only if configuration was changed.

Copy Export

- systemctl restart httpd

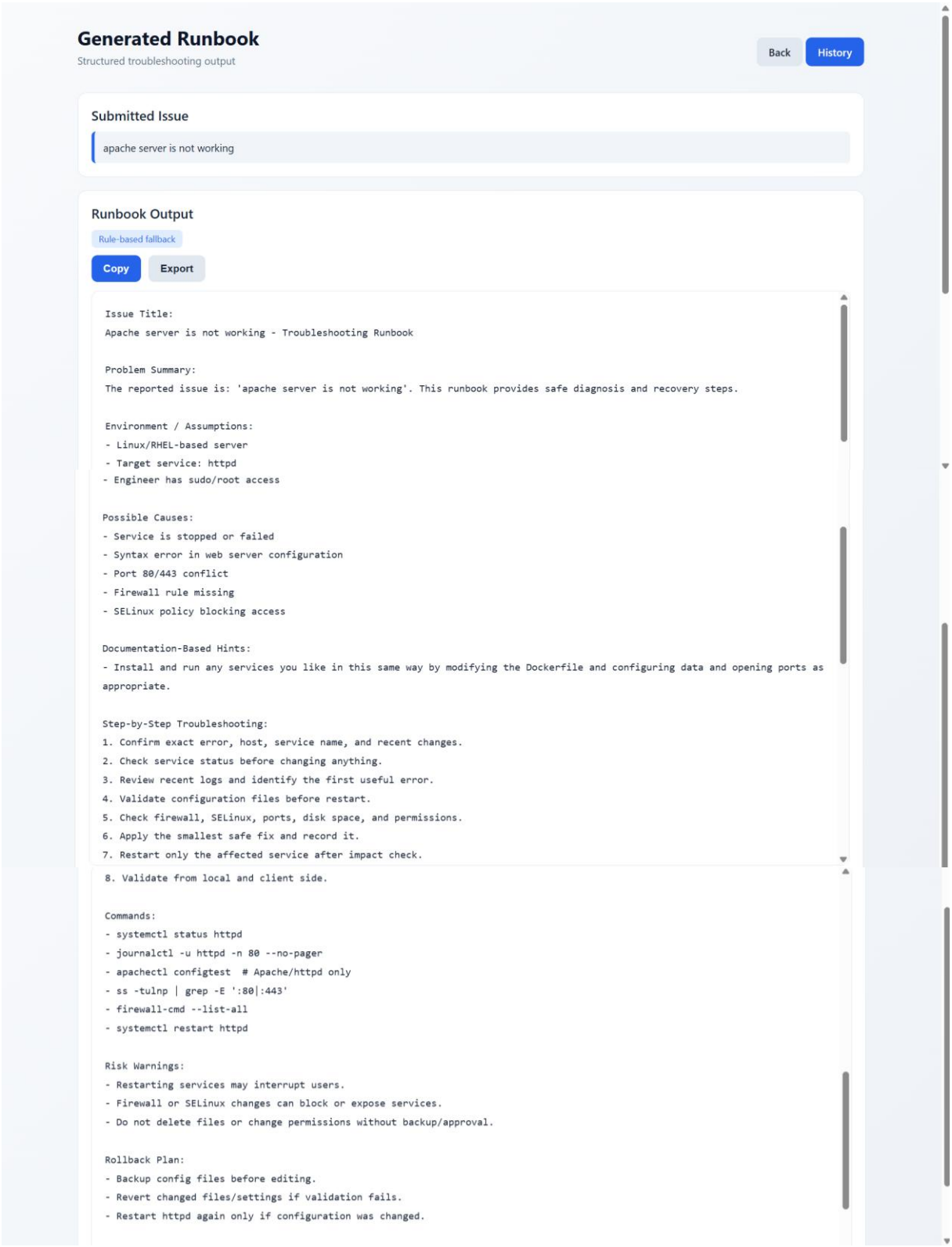
Risk Warnings:
- Restarting services may interrupt users.
- Firewall or SELinux changes can block or expose services.
- Do not delete files or change permissions without backup/approval.

Rollback Plan:
- Backup config files before editing.
- Revert changed files/settings if validation fails.
- Restart httpd again only if configuration was changed.

Validation Steps:
- Confirm 'apache server is not working' is resolved.
- Run systemctl status httpd if applicable.
- Recheck logs for new critical errors.
- Test from user/client side.

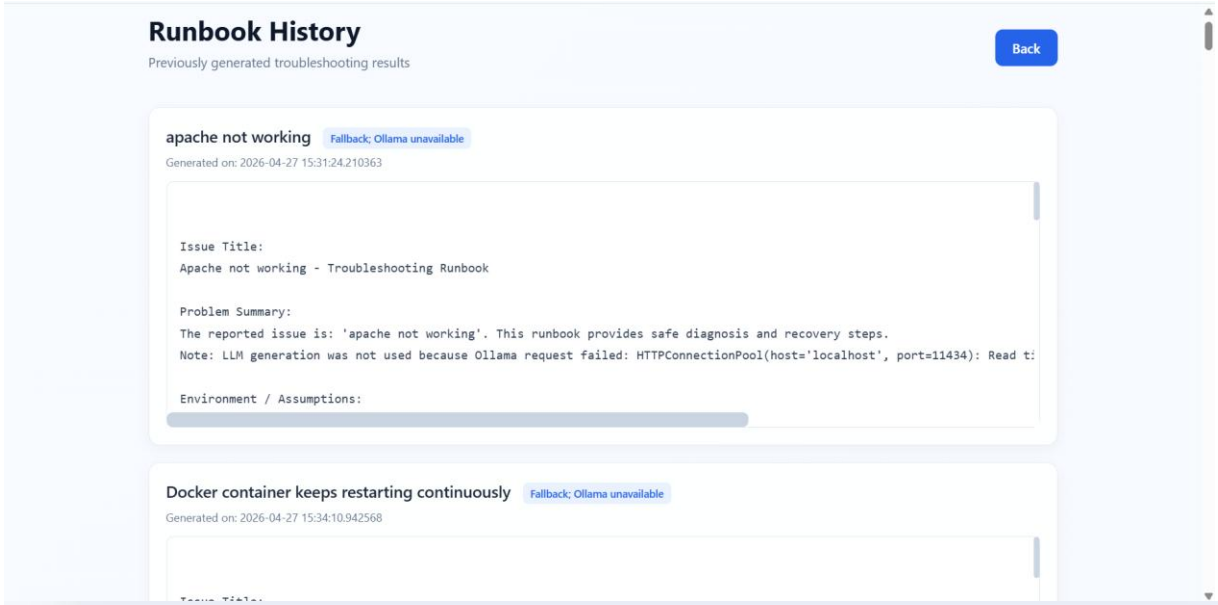
Escalation Notes:
- Escalate if issue repeats, affects multiple servers, or shows kernel/storage/security errors.
- Attach logs, command outputs, recent changes, and documentation references.

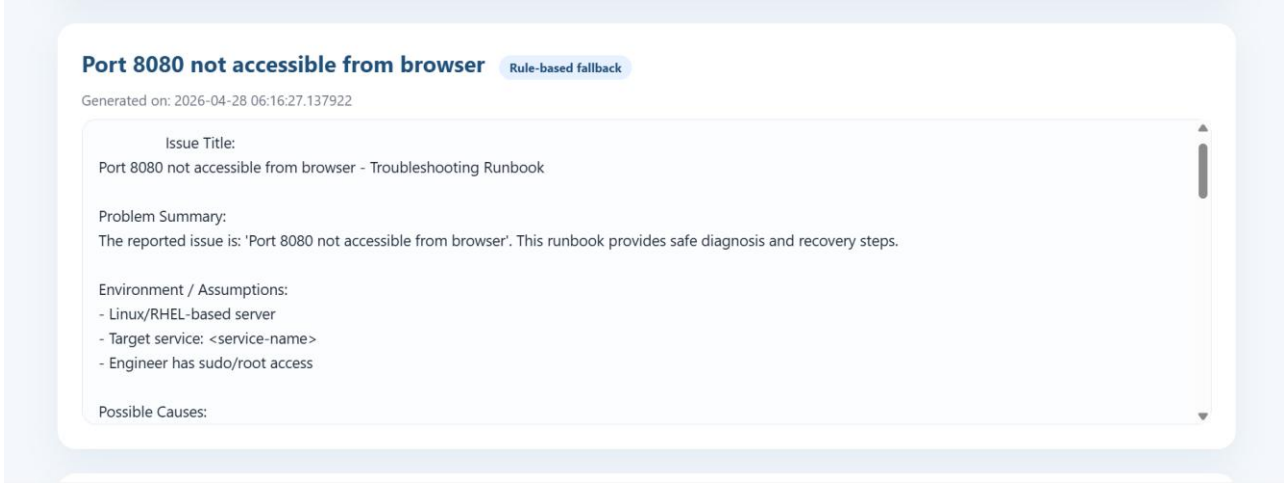
Step 6: Display Output



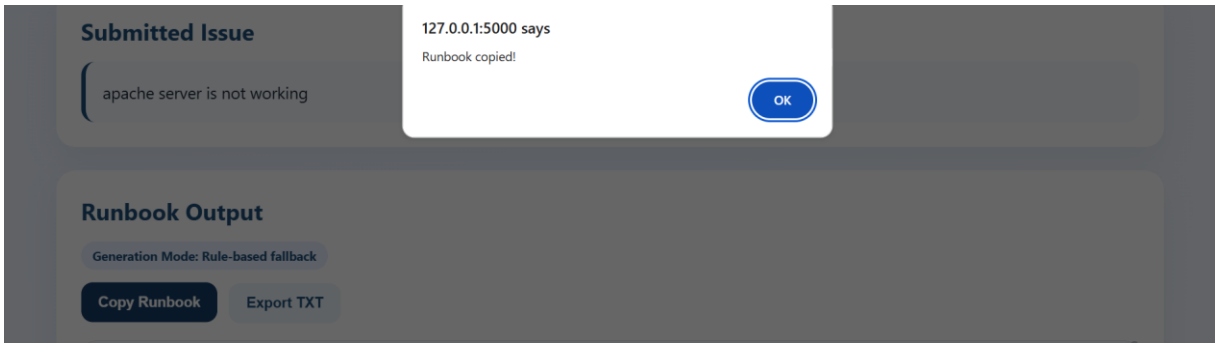


Step 7: Store in History



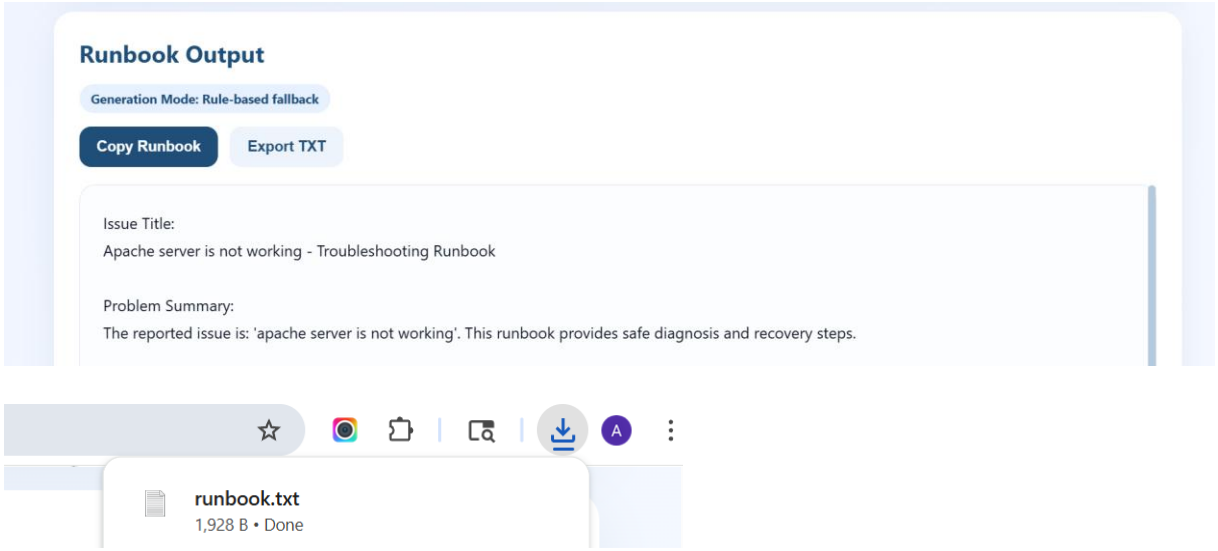


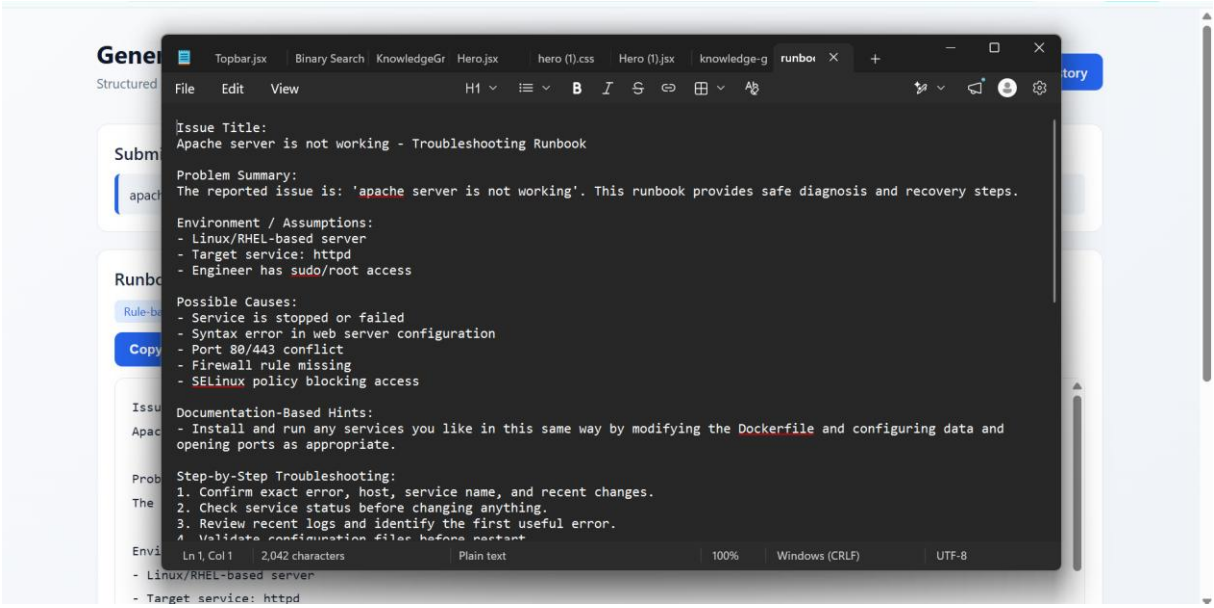
Step 8: Copy Runbook



Step 9: Export Runbook

The user can download the runbook as a text file for documentation or sharing purposes.





Implementation of RunbookGen AI

The implementation of RunbookGen AI is divided into multiple Python and HTML files. Each file performs a specific role in the system such as routing, retrieval, runbook generation, database handling, prompt creation, and user interface design.

1. app.py – **Main Flask Application**

This is the main backend file of the project. It initializes the Flask application, manages routes, handles login/logout, processes user input, generates runbooks, stores history, and supports export functionality.

- /login route

```python
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")

        if username == USERNAME and password == PASSWORD:
            session["logged_in"] = True
            return redirect(url_for("index"))
        else:
            return render_template("login.html", error="Invalid username or password")

    return render_template("login.html")

```

- /logout route

```python
@app.route("/logout")
def logout():
    session.clear()
    return redirect(url_for("login"))


@app.route("/", methods=["GET"])
def index():
    if not session.get("logged_in"):
        return redirect(url_for("login"))

    return render_template("index.html")

```

- /generate route

```python
@app.route("/generate", methods=["POST"])
def generate():
    if not session.get("logged_in"):
        return redirect(url_for("login"))

    query = request.form.get("issue", "").strip()
    use_llm = False

    if not query:
        return render_template(
            "index.html",
            error="Please enter a system issue, for example: Apache service not starting on RHEL.",
        )

    try:
        docs = retrieve_documents(query, top_k=3, max_results=5)
        result, mode = generate_runbook(query, docs, use_llm=use_llm)

        history = RunbookHistory(issue=query, result=result, mode=mode)
        db.session.add(history)
        db.session.commit()

    except Exception as exc:
        docs = []
        mode = "Error"
        result = f"Error while generating runbook: {exc}"

    return render_template("result.html", query=query, result=result, docs=docs, mode=mode)

```

- /history route

```python
@app.route("/history")
def history():
    if not session.get("logged_in"):
        return redirect(url_for("login"))

    items = RunbookHistory.query.order_by(RunbookHistory.created_at.desc()).all()
    return render_template("history.html", items=items)

```

- /export route

```python
@app.route("/export", methods=["POST"])
def export():
    if not session.get("logged_in"):
        return redirect(url_for("login"))

    content = request.form.get("content", "")
    return Response(
        content,
        mimetype="text/plain",
        headers={"Content-Disposition": "attachment;filename=runbook.txt"}
    )

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True, use_reloader=False)
```

2. **models.py – Database Model**

This file defines the database structure using SQLAlchemy. It creates the RunbookHistory table to store the user issue, generated result, generation mode, and timestamp.

models.py

```python
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime

db = SQLAlchemy()

class RunbookHistory(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    issue = db.Column(db.Text, nullable=False)
    result = db.Column(db.Text, nullable=False)
    mode = db.Column(db.String(100), default="Unknown")
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

3. **retriever.py – Document Retrieval Module**

This file handles semantic document retrieval. It loads the FAISS index, metadata, and sentence-transformer model. The user query is converted into embeddings and matched with stored documentation to retrieve relevant content.

- load_resources()

```python
import os
import pickle
from typing import Dict, List

import faiss
from sentence_transformers import SentenceTransformer

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
INDEX_PATH = os.path.join(BASE_DIR, "data", "redhat_docs.index")
METADATA_PATH = os.path.join(BASE_DIR, "data", "chunk_metadata.pkl")

_model = None
_index = None
_metadata = None


def load_resources():
    global _model, _index, _metadata

    if _model is None:
        _model = SentenceTransformer("all-MiniLM-L6-v2")

    if _index is None:
        if not os.path.exists(INDEX_PATH):
            raise FileNotFoundError(f"FAISS index not found at {INDEX_PATH}")
        _index = faiss.read_index(INDEX_PATH)

    if _metadata is None:
        if not os.path.exists(METADATA_PATH):
            raise FileNotFoundError(f"Metadata file not found at {METADATA_PATH}")
        with open(METADATA_PATH, "rb") as f:
            _metadata = pickle.load(f)
```



```python
    return _model, _index, _metadata


# ✅ PRELOAD (OUTSIDE FUNCTION)
load_resources()


def clean_text(text: str) -> str:
    return " ".join(str(text).replace("\n", " ").split())


def is_useful_text(text: str) -> bool:
    if not isinstance(text, str):
        return False

    text = clean_text(text)

    if len(text) < 80:
        return False

    noisy_tokens = ["noarch", ".rpm", "redhat_", "x86_64", "package list", "copyright"]

    if sum(token in text.lower() for token in noisy_tokens) >= 2:
        return False

    return True
```

- retrieve_documents()

```python
def retrieve_documents(query: str, top_k: int = 8, max_results: int = 4) -> List[Dict[str, object]]:
    model, index, metadata = load_resources()

    query_embedding = model.encode([query], normalize_embeddings=True).astype("float32")
    distances, indices = index.search(query_embedding, top_k)

    results = []
    seen = set()

    for rank, idx in enumerate(indices[0], start=1):
        if not (0 <= idx < len(metadata)):
            continue

        item = metadata[idx]

        if isinstance(item, dict):
            text = item.get("chunk_text") or item.get("text") or str(item)
            source = item.get("source") or item.get("title") or "Red Hat documentation"
        else:
            text = str(item)
            source = "Red Hat documentation"

        text = clean_text(text)
        fingerprint = text[:180]

        if fingerprint in seen or not is_useful_text(text):
            continue

        seen.add(fingerprint)

        results.append({
            "rank": rank,
            "score": float(distances[0][rank - 1]),
            "source": source,
            "text": text[:1200]
        })

        if len(results) >= max_results:
            break

    return results
```

4. generator.py – Runbook Generation Logic

This file contains the main logic for generating troubleshooting runbooks. It detects the issue type, selects suitable commands, extracts useful hints from retrieved documents, and generates structured runbooks with sections like causes, steps, commands, risk warnings, rollback plan, and validation steps.

generator.py

```python
import re
from typing import Dict, List, Tuple

from llm_client import LLMError, generate_with_ollama
from prompt_template import build_prompt


SERVICE_HINTS = {
    "apache": "httpd",
    "httpd": "httpd",
    "nginx": "nginx",
    "mysql": "mysqld",
    "mariadb": "mariadb",
    "docker": "docker",
    "podman": "podman",
    "ssh": "sshd",
    "firewall": "firewalld",
    "firewalld": "firewalld",
}


REQUIRED_SECTIONS = [
    "Issue Title:",
    "Problem Summary:",
    "Environment / Assumptions:",
    "Possible Causes:",
    "Step-by-Step Troubleshooting:",
    "Commands:",
    "Risk Warnings:",
    "Rollback Plan:",
    "Validation Steps:",
    "Escalation Notes:",
```

```python

def detect_service(query: str) -> str:
    q = query.lower()
    for key, service in SERVICE_HINTS.items():
        if key in q:
            return service
    return "<service-name>"

```

- detect_issue_type()

generator.py

```python
def detect_issue_type(query: str) -> str:
    q = query.lower()

    if any(word in q for word in ["apache", "httpd", "nginx", "web server"]):
        return "web_service"
    if any(word in q for word in ["docker", "container", "podman"]):
        return "container"
    if any(word in q for word in ["selinux", "permission denied", "avc"]):
        return "selinux"
    if any(word in q for word in ["port", "firewall", "connection refused", "network"]):
        return "network"
    if any(word in q for word in ["disk", "space", "storage", "mount"]):
        return "storage"

    return "general"


def extract_doc_hints(docs: List[Dict[str, object]]) -> List[str]:
    hints = []

    keywords = [
        "systemctl",
        "journalctl",
        "firewall-cmd",
        "getenforce",
        "setsebool",
        "semanage",
        "restorecon",
        "ss -",
        "curl",
        "podman",
        "docker",
        "apachectl",
    ]

    for doc in docs:
        text = doc.get("text", "") if isinstance(doc, dict) else str(doc)
        sentences = re.split(r"(?<=[.!?])\s+", text)

        for sentence in sentences:
            lower = sentence.lower()

            if any(k in lower for k in keywords) and len(sentence) < 240:
                clean = sentence.strip(" -•\t")

                if clean and clean not in hints:
                    hints.append(clean)

            if len(hints) >= 4:
                return hints

    return hints
```

- build_commands()

```python
def build_commands(issue_type: str, service: str) -> List[str]:
    common = [
        f"systemctl status {service}",
        f"journalctl -u {service} -n 80 --no-pager",
    ]

    if issue_type == "web_service":
        return common + [
            "apachectl configtest  # Apache/httpd only",
            "ss -tulnp | grep -E ':80|:443'",
            "firewall-cmd --list-all",
            f"systemctl restart {service}",
        ]

    if issue_type == "container":
        return [
            "docker ps -a  # or: podman ps -a",
            "docker logs <container_id>",
            "docker inspect <container_id>",
            "ss -tulnp",
            "docker restart <container_id>",
        ]

    if issue_type == "selinux":
        return common + [
            "getenforce",
            "ausearch -m avc -ts recent",
            "restorecon -Rv <affected_path>",
            "semanage fcontext -l | grep <path_or_service>",
        ]
    if issue_type == "network":
        return common + [
            "ss -tulnp",
            "firewall-cmd --list-all",
            "curl -I http://localhost",
            "ping <server_ip>",
        ]

    if issue_type == "storage":
        return [
            "df -h",
            "du -sh <path>",
            "lsblk",
            "mount | column -t",
            "journalctl -xe --no-pager",
        ]

    return common + [
        "journalctl -xe --no-pager",
        "ss -tulnp",
        "df -h",
        f"systemctl restart {service}",
    ]

```

- generate_runbook()

generator.py

```python

def generate_runbook(
    query: str,
    docs: List[Dict[str, object]],
    use_llm: bool = True
) -> Tuple[str, str]:
    """Return (runbook_text, generation_mode)."""

    prompt = build_prompt(query, docs)

    if use_llm:
        try:
            answer = generate_with_ollama(prompt)

            if validate_output(answer):
                return answer, "Ollama GenAI + RAG"

            return (
                fallback_runbook(query, docs),
                "Fallback after LLM validation",
            )

        except LLMError:
            return (
                fallback_runbook(query, docs),
                "Fallback; Ollama unavailable",
            )

    return fallback_runbook(query, docs), "Rule-based fallback"
```

5. prompt_template.py – Prompt Building Module

This file creates a structured prompt for AI-based runbook generation. It combines the user query with retrieved documentation and defines the required output format.

- build_prompt()

prompt_template.py

```python
def build_prompt(query, docs):
    context_blocks = []
    for i, doc in enumerate(docs, start=1):
        if isinstance(doc, dict):
            text = doc.get("text", "")
            source = doc.get("source", "Red Hat documentation")
            score = doc.get("score", "")
            context_blocks.append(f"[Doc {i} | Source: {source} | Score: {score}]\n{text}")
        else:
            context_blocks.append(f"[Doc {i}]\n{doc}")

    context = "\n\n".join(context_blocks) if context_blocks else "No matching documentation was retri

    return f"""
```

- Required output format section

prompt_template.py

```
You are RunbookGen AI, a senior Linux and Red Hat support engineer.
Create a production-ready troubleshooting runbook for a junior engineer.

User issue:
{query}

Retrieved documentation context:
{context}

Strict rules:
- Use the retrieved documentation only when it is relevant to the issue.
- Do not copy long raw documentation text.
- Do not invent destructive commands.
- Prefer safe diagnostic commands before restart, deletion, permission, firewall, or SELinux changes.
- Mention impact before service restart, firewall change, SELinux change, or configuration change.
- Use simple professional language.
- Keep commands realistic for Linux/RHEL.
- If the exact service name is unknown, use placeholders like <service-name>.

Required output format exactly:

Issue Title:

Problem Summary:

Environment / Assumptions:

Possible Causes:

Step-by-Step Troubleshooting:
1.
2.
3.
4.
5.

Commands:

Risk Warnings:

Rollback Plan:

Validation Steps:

Escalation Notes:
"""
```

6. llm_client.py – Optional LLM Integration

This file connects the system with the Ollama local LLM API. It sends the generated prompt to the model and receives an AI-generated response. If the model is unavailable, the system can use fallback logic.

- Ollama configuration

llm_client.py

```
"""LLM client for RunbookGen AI.

Default provider: Ollama running locally.
Fallback: deterministic rule-based generator if Ollama is unavailable.
"""

import os
import requests

OLLAMA_URL = os.getenv("OLLAMA_URL", "http://localhost:11434/api/generate")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.2:3b")
REQUEST_TIMEOUT = int(os.getenv("OLLAMA_TIMEOUT", "120"))


class LLMError(RuntimeError):
    pass
```

- generate_with_ollama()

```python
def generate_with_ollama(prompt: str) -> str:
    payload = {
        "model": OLLAMA_MODEL,
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": 0.2,
            "top_p": 0.9,
            "num_ctx": 4096,
        },
    }

    try:
        response = requests.post(OLLAMA_URL, json=payload, timeout=REQUEST_TIMEOUT)
        response.raise_for_status()
    except requests.RequestException as exc:
        raise LLMError(f"Ollama request failed: {exc}") from exc

    data = response.json()
    answer = data.get("response", "").strip()
    if not answer:
        raise LLMError("Ollama returned an empty response.")
    return answer
```

Frontend Implementation

7. login.html – Login Page

This file provides the login interface for users. It includes username and password fields, error message display, and a clean card-style design.

templates > login.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login - RunbookGen AI</title>

    <style>
        * {
            box-sizing: border-box;
        }

        body {
            margin: 0;
            font-family: "Segoe UI", Arial, sans-serif;
            background: linear-gradient(135deg, #eef4ff, #f7fbff);
            color: #1f2937;
            min-height: 100vh;
            display: flex;
            align-items: center;
            justify-content: center;
            padding: 20px;
        }

        .login-card {
            width: 100%;
            max-width: 430px;
            background: white;
            border-radius: 24px;
            padding: 38px;
            box-shadow: 0 20px 50px rgba(31, 78, 121, 0.14);
        }
```

```
.badge {
    display: inline-block;
    background: #e8f1ff;
    color: #1f4e79;
    padding: 8px 14px;
    border-radius: 999px;
    font-size: 14px;
    font-weight: 700;
    margin-bottom: 18px;
}

h1 {
    margin: 0 0 10px;
    color: #123a63;
    font-size: 34px;
}

p {
    color: #5b6472;
    line-height: 1.6;
    margin-bottom: 24px;
}

label {
    display: block;
    font-weight: 700;
    margin-bottom: 8px;
    color: #163d66;
}

input {
    width: 100%;
    padding: 14px 16px;
    border-radius: 14px;
    border: 1px solid #cbd5e1;
    font-size: 15px;
    margin-bottom: 18px;
    outline: none;
}

input:focus {
    border-color: #1f4e79;
    box-shadow: 0 0 0 4px rgba(31, 78, 121, 0.1);
}

button {
    width: 100%;
    border: none;
    background: #1f4e79;
    color: white;
    padding: 14px;
    border-radius: 14px;
    font-size: 16px;
    font-weight: 700;
    cursor: pointer;
    transition: 0.2s;
}
```

```
        button:hover {
            background: #163d66;
            transform: translateY(-1px);
        }

        .error {
            background: #fee2e2;
            color: #991b1b;
            padding: 12px;
            border-radius: 12px;
            margin-bottom: 18px;
            font-weight: 600;
        }

        .hint {
            margin-top: 18px;
            background: #f6faff;
            border: 1px solid #dce9f8;
            padding: 12px;
            border-radius: 14px;
            font-size: 14px;
            color: #374151;
        }
    </style>
</head>

<body>
    <div class="login-card">
        <div class="badge">Secure Access</div>

        <h1>RunbookGen AI</h1>
        <p>Login to access the AI-powered IT troubleshooting runbook generator.</p>

        {% if error %}
            <div class="error">{{ error }}</div>
        {% endif %}

        <form method="POST" action="/login">
            <label for="username">Username</label>
            <input type="text" id="username" name="username" placeholder="Enter username" required>

            <label for="password">Password</label>
            <input type="password" id="password" name="password" placeholder="Enter password" require

            <button type="submit">Login</button>
        </form>

        <div class="hint">
            Demo Login: <b>admin</b> / <b>admin123</b>
        </div>
    </div>
</body>
</html>
```

8. **index.html – Home/Input Page**

This file acts as the main input page after login. It allows users to enter an IT issue and submit it for runbook generation. It also provides access to the history page.

```
</head>
<body>
    <section class="hero">
        <div class="card">
            <div class="badge">Enterprise IT Automation</div>
            <h1>RunbookGen AI</h1>
            <p class="subtitle">
                Generate structured troubleshooting runbooks from issue descriptions using
                enterprise documentation, intelligent retrieval, and clean automated formatting.
            </p>

            <div class="feature-row">
                <div class="feature-box">
                    <div class="feature-title">Smart Retrieval</div>
                    <div>Finds the most relevant enterprise troubleshooting context.</div>
                </div>
                <div class="feature-box">
                    <div class="feature-title">Structured Output</div>
                    <div>Creates steps, commands, warnings, and validation checks.</div>
                </div>
                <div class="feature-box">
                    <div class="feature-title">Professional UI</div>
                    <div>Clean interface for project demos and academic presentation.</div>
                </div>
            </div>

            <form action="/generate" method="POST">
                <label for="issue">Describe the IT issue</label>
                <textarea
                    id="issue"
                    name="issue"
                    placeholder="Example: Apache server is not working after a configuration update,
                    required
                ></textarea>

                <div class="btn-row">
    <button class="btn" type="submit">Generate Runbook</button>
    <button class="btn btn-light" type="reset">Clear</button>
    <a href="/history" class="btn btn-light">History</a>
                </div>
            </form>
        </div>
    </section>
</body>
</html>
```

9. result.html – Generated Runbook Page

This file displays the generated runbook output. It shows the submitted issue, generation mode, runbook result, retrieved documentation context, copy button, and export option.

```
<body>
    <div class="wrapper">

        <div class="topbar">
            <div class="title-box">
                <h1>Generated Runbook</h1>
                <p>Structured troubleshooting output generated for the submitted issue.</p>
            </div>

            <div class="action-row">
                <a href="/" class="btn btn-light">← Back to Home</a>
                <a href="/history" class="btn">View History</a>
            </div>
        </div>

        <div class="card">
            <h2 class="section-title">Submitted Issue</h2>
            <div class="query-box">{{ query }}</div>
        </div>

        <div class="card">
            <h2 class="section-title">Runbook Output</h2>

            {% if mode %}
                <div class="badge">Generation Mode: {{ mode }}</div>
            {% endif %}

            <div class="action-row">
                <button onclick="copyRunbook()" class="btn">Copy Runbook</button>

                <form action="/export" method="POST">
```

```
                    <textarea name="content" hidden>{{ result }}</textarea>
                    <button type="submit" class="btn btn-light">Export TXT</button>
                </form>
            </div>

            <div class="runbook-box" id="runbookText">{{ result }}</div>
        </div>

        <div class="card">
            <h2 class="section-title">Retrieved Documentation Context</h2>

            <div class="context-grid">
                {% for doc in docs %}
                    <div class="context-item">
                        {% if doc.text %}
                            {{ doc.text }}
                        {% else %}
                            {{ doc }}
                        {% endif %}
                    </div>
                {% else %}
                    <div class="empty-text">No documentation context was retrieved.</div>
                {% endfor %}
            </div>
        </div>

```

Testing and Results

The RunbookGen AI system was tested using multiple real-world IT troubleshooting scenarios to evaluate its functionality, accuracy, and performance. Different types of issues related to web services, system processes, and network configurations were provided as input through the user interface.
The testing focused on verifying:

- Correct processing of user input
- Accurate retrieval of relevant documentation
- Proper generation of structured runbooks
- Storage of results in the database
- Functionality of export and history features

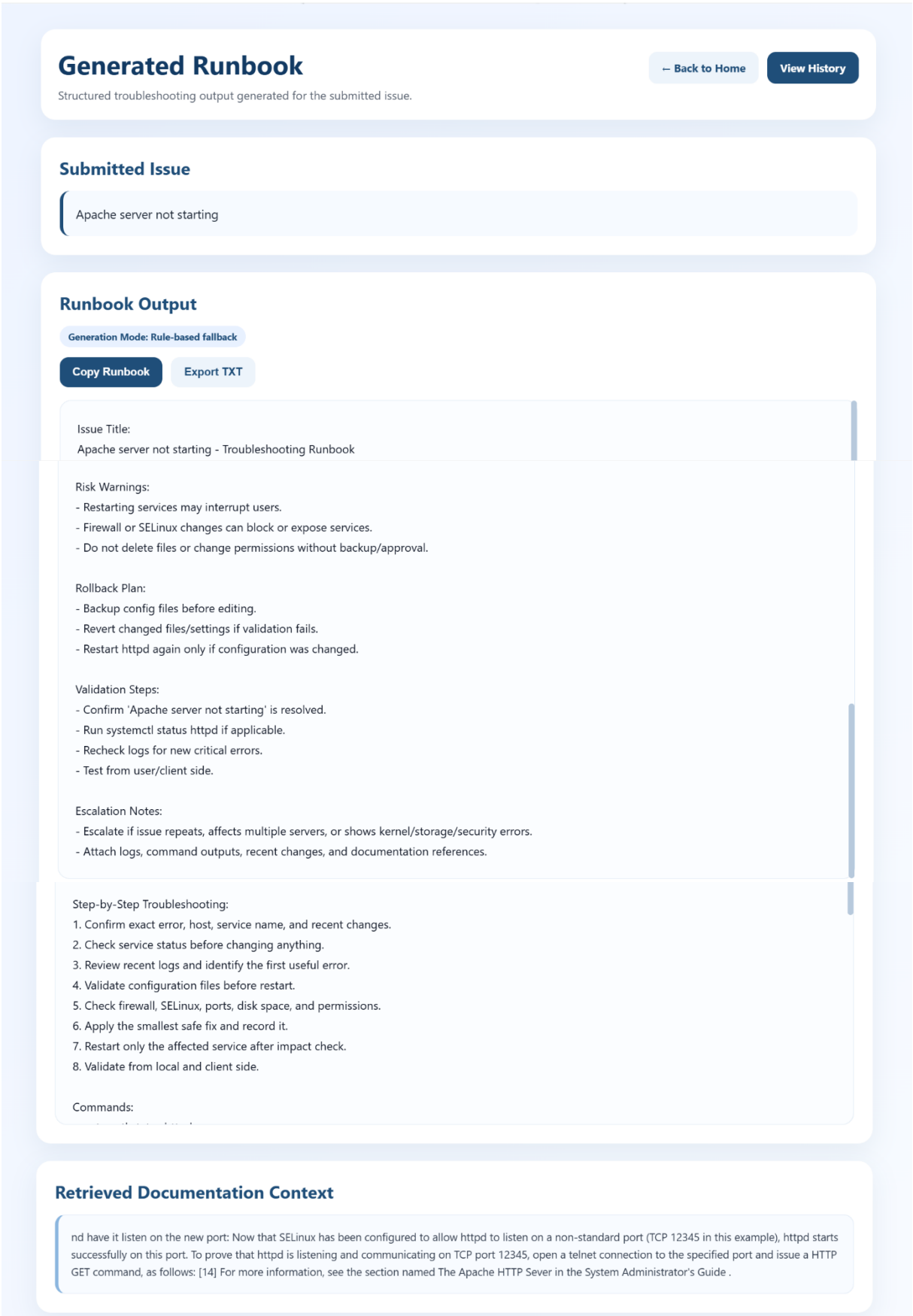
Test Cases Used:

The following test cases were used to evaluate the system:

- Apache server not starting
- Service not running or failed state
- Port not accessible / connection refused
- General system troubleshooting queries

Results Obtained:

- The system successfully accepted user inputs through the interface and processed them without errors
- The document retrieval module returned relevant context using FAISS and embeddings
- The runbook generation engine produced structured outputs including:
  1. Problem Summary
  2. Possible Causes
  3. Step-by-step Troubleshooting
  4. Commands
  5. Risk Warnings
  6. Validation Steps
- The generated runbooks were clear, consistent, and easy to understand
- The system correctly identified the issue type and suggested appropriate commands (e.g., systemctl, journalctl, ss, etc.)
- All generated runbooks were stored in the SQLite database and displayed correctly in the history page
- The export feature worked successfully, allowing users to download runbooks as text files



Performance Evaluation

- The system responded quickly to user queries with minimal delay
- Retrieval of documents was efficient due to FAISS indexing
- The rule-based logic ensured consistent and reliable output
- The user interface functioned smoothly without navigation issues

Overall Observation
The system performed effectively across all tested scenarios. It was able to generate structured and meaningful troubleshooting solutions for different types of IT issues. The combination of retrieval-based context and rule-based logic improved the accuracy and usefulness of the generated runbooks.

Future Enhancements

- Integration with advanced AI models (GenAI)
- Voice-based input system
- Cloud deployment for scalability
- Real-time log analysis
- Multi-user authentication system
- Advanced analytics dashboard

Conclusion

RunbookGen AI is an effective solution for automating IT troubleshooting by generating structured and easy-to-understand runbooks from user-reported issues. The system simplifies the problem-solving process by converting natural language inputs into organized troubleshooting steps, including causes, commands, and validation procedures.

By integrating document retrieval techniques with rule-based logic, the system ensures that the generated solutions are relevant, consistent, and practical. It reduces the time and effort required for manual debugging while providing a standardized approach that is especially useful for beginners and junior engineers.

The implementation using Flask, FAISS, and SQLite demonstrates a modular and efficient system design. Features such as history tracking and export functionality further enhance usability and support knowledge reuse.

Overall, RunbookGen AI improves efficiency, minimizes human error, and provides a reliable tool for IT support and learning. With future enhancements such as full AI integration and advanced analysis capabilities, the system has the potential to evolve into a comprehensive intelligent troubleshooting assistant for real-world applications.